Git to Render Deployment Guide

Taking a Django project from your computer to a live website, step by step

Project: Virtual Bank of India | Stack: Django, PostgreSQL, Git, GitHub, Render

What you will achieve

This guide records the complete process used to publish the Virtual Bank of India project: preparing the code, saving it with Git, uploading it to GitHub, and deploying it automatically on Render. Follow the steps in order. Each step explains what to do, the exact command, what you should see, and why it matters.

Live site: **https://virtual-bank-of-india.onrender.com/**

The flow at a glance

1. Prepare Project folder and deployment files	**2. Git** Install, identify, init, commit	**3. GitHub** Create repository and push	**4. Render** Blueprint deploy and verify

Before you start

- A Windows computer with the project working locally (all commands below are for PowerShell).
- Git installed. Check with `git --version`.
- A free GitHub account (github.com) and a free Render account (render.com).
- Your Django project folder, the one that contains `manage.py`.

Part 1: Prepare the project

1 Find the correct project folder

Render and GitHub expect `manage.py`, `build.sh` and `render.yaml` at the top level of the repository. The Git repository must therefore be created inside the folder that contains `manage.py`, not in a parent folder.

In this project that folder is:

```
D:\NewProject\Virtual Bank Of India\django_banking
```

A correct layout looks like this:

```
django_banking/
  manage.py
  build.sh
  render.yaml
  requirements.txt
  .gitignore
  .gitattributes
  .env.example
  README.md
  bank/            (the app)
  banking_project/ (settings, urls, wsgi)
  templates/
```

Mistake to avoid: running `git init` in a higher folder such as `D:\NewProject`. The repository then contains extra folders and Render cannot find `manage.py`. If this happens, delete the hidden `.git` folder

and start again in the right place (see Step 5).

2 Add the deployment files

These files tell Render how to build and run the app. Check that each exists in the project folder.

File	Purpose
requirements.txt	Lists the Python packages to install (Django, gunicorn, psycopg for PostgreSQL, whitenoise for static files, and so on).
build.sh	The build script Render runs: installs packages, collects static files, and applies database migrations.
render.yaml	The Blueprint. Describes the web service and the PostgreSQL database so Render can create both in one click.
.env.example	A sample of the settings the app needs, with fake values. The real `.env` is never uploaded.
.gitignore	Tells Git to skip secrets and clutter: `.env`, `venv/`, `__pycache__/`, `db.sqlite3`.
.gitattributes	Keeps Linux line endings on `build.sh` (explained in Step 4).
.python-version	Pins the Python version Render uses.
README.md	The project description shown on the GitHub page.

A typical `build.sh` looks like this (your file may differ slightly):

```
#!/usr/bin/env bash
set -o errexit

pip install -r requirements.txt
python manage.py collectstatic --no-input
python manage.py migrate
```

Never upload your real `.env` file. It holds the secret key and database password. Only `.env.example` with placeholder values belongs in the repository.

Part 2: Git on your computer

3 Tell Git who you are

Git stamps every commit with a name and email. Without them the first commit fails with *Author identity unknown*. This is a one-time setup per computer.

```
git config --global user.name "Pravin Bansode"
git config --global user.email "pravinbansode146@gmail.com"
```

Use the same email that is registered on your GitHub account so the commits are linked to your profile. Check the saved values with `git config --global --list`.

4 Keep Linux line endings for build.sh

Windows saves line breaks as CRLF, but Render runs Linux, which needs LF. If `build.sh` keeps CRLF, the build fails with `/usr/bin/env: 'bash\r': No such file or directory`. A `.gitattributes` file prevents this.

Run this inside the project folder:

```
cd "D:\NewProject\Virtual Bank Of India\django_banking"
"* text=auto`n*.sh text eol=lf" | Out-File -Encoding ascii .gitattributes
```

Warnings such as *LF will be replaced by CRLF* that appear later are harmless. Git is only noting the Windows and Linux difference.

5 Create the repository in the right folder

If you earlier ran `git init` in the wrong folder, remove that setup first. Your project files are not deleted, only the hidden `.git` folder.

```
cd D:\NewProject
Remove-Item -Recurse -Force .git
```

Now create the repository where `manage.py` lives:

```
cd "D:\NewProject\Virtual Bank Of India\django_banking"
git init
git add .
git status
```

`git add .` stages every file for the first commit. `git status` lists what is staged so you can check it before saving.

6 Check the file list before committing

Look at the green *new file* lines and confirm:

- `manage.py`, `build.sh`, `render.yaml` and `requirements.txt` appear directly, not inside another folder.
- There is **no** `.env` file in the list.
- There is **no** `venv` folder in the list.
- Python package folders contain `__init__.py` with **two** underscores on each side. A name with single underscores is wrong and Django will not treat the folder as a package.

If the list looks wrong, stop and fix it before continuing. It is much easier now than after pushing.

7 Make the first commit

```
git commit -m "Virtual Bank Of India"
```

A successful commit prints `[master (root-commit) ...]` and the number of files changed. This project saved 34 files.

8 Mark build.sh as executable

Windows does not track the executable permission, so Git stores `build.sh` as mode 100644. Render then fails with *Permission denied*. Mark the file as executable inside Git:

```
git update-index --chmod=+x build.sh
git commit -m "Make build.sh executable"
```

The output should include `mode change 100644 => 100755 build.sh`.

Part 3: GitHub

9 Create an empty repository on GitHub

- Sign in at **github.com** and open **github.com/new**.
- Repository name: `virtual-bank-of-india`.
- Choose Public (recommended for a portfolio) or Private. Render works with either.
- Leave **Add a README**, **.gitignore** and **license** unchecked. A non-empty repository causes conflicts on the first push.
- Click **Create repository**.

Your username is shown in the top-right menu and in the address bar, for example `github.com/your-username/virtual-bank-of-india`.

10 Connect your folder to GitHub and push

Rename the branch to `main`, add the remote address, and upload. Replace `REAL_USERNAME` with your own GitHub username and type the address as plain text.

```
git branch -M main
git remote add origin https://github.com/REAL_USERNAME/virtual-bank-of-india.git
git push -u origin main
```

When a window opens, choose **Sign in with your browser** and approve the login. A successful push ends with `* [new branch] main -> main`.

Repository not found? The address still contains the placeholder `YOUR_USERNAME`, or the repository name is misspelled. Fix the address and push again:

```
git remote set-url origin https://github.com/REAL_USERNAME/virtual-bank-of-india.git
git remote -v
git push -u origin main
```

`git remote -v` should show your real username twice.

11 Verify the repository on GitHub

Refresh the repository page. You should see `manage.py`, `build.sh`, `render.yaml` and the app folders at the top, your README below them, and no `.env` file.

Part 4: Deploy on Render

12 Sign in to Render with GitHub

Go to **render.com** and choose **Sign in with GitHub**. When asked, allow Render to access your repositories (either all, or just `virtual-bank-of-india`).

13 Create the services from the Blueprint

- In the Render dashboard click **New**, then **Blueprint**.
- Select the `virtual-bank-of-india` repository.
- Render reads `render.yaml` and lists the web service and PostgreSQL database it will create.
- Click **Apply**.

Render now runs `build.sh`, starts the app, and connects it to the database. The first build takes about 3 to 5 minutes. Watch progress under **Logs**; a finished build ends with a message that the service is live.

14 Open and test the live site

Open the web service page and click the address ending in `.onrender.com`. For this project it is:

https://virtual-bank-of-india.onrender.com/

Test the main features: open a demo account, log in, make a deposit and a withdrawal, and check that the transaction history updates.

> On Render's free plan the site goes to sleep after a period of inactivity, and the first visit afterwards can take 30 to 60 seconds to wake up. Open the link yourself shortly before sharing it with an interviewer.

15 Update the site later

Render redeploys automatically each time you push to the `main` branch:

```
git add .
git commit -m "Describe your change"
git push
```

Check the Render **Logs** tab to confirm the new build succeeded.

16 Show it on your GitHub profile

- Open your profile page and click **Customize your pins**.
- Tick `virtual-bank-of-india` and save.
- On the repository page, click the gear icon beside **About**, paste the live site address into **Website**, and add topics such as django, python and postgresql.

Troubleshooting

Problem	Cause	Fix
LF will be replaced by CRLF warning	Windows and Linux use different line endings.	Harmless. Ignore, as long as `.gitattributes` exists.
Author identity unknown	Git does not know your name and email.	Run the two `git config --global` commands in Step 3.
Repository not found on push	Placeholder or wrong username or repository name in the remote address.	Run `git remote set-url origin ...` with the correct address (Step 10).
Render cannot find manage.py	Repository was created in a parent folder.	Remove `.git` and re-create it inside the project folder (Steps 1 and 5).
'bash\r': No such file or directory	`build.sh` has Windows line endings.	Add `.gitattributes` (Step 4), run `git add --renormalize .`, commit and push.
Permission denied on build.sh	File is not marked executable.	Run `git update-index --chmod=+x build.sh`, commit and push (Step 8).
Django cannot import the app or settings	`__init__.py` misnamed with single underscores.	Rename to `__init__.py`, then add, commit and push.
Build fails on Render	Missing package, wrong Python version, or missing environment variable.	Read the last lines of the Render **Logs**, fix the cause locally, and push again.
Site is slow on first visit	Free plan service was asleep.	Wait up to a minute, then refresh.

Quick command summary

```
cd "D:\NewProject\Virtual Bank Of India\django_banking"
git config --global user.name "Pravin Bansode"
git config --global user.email "pravinbansode146@gmail.com"
"* text=auto`n*.sh text eol=lf" | Out-File -Encoding ascii .gitattributes
git init
git add .
git status
git commit -m "Virtual Bank Of India"
git update-index --chmod=+x build.sh
git commit -m "Make build.sh executable"
git branch -M main
git remote add origin https://github.com/REAL_USERNAME/virtual-bank-of-india.git
git push -u origin main
```

Then on Render: New, Blueprint, select the repository, Apply. Live site: https://virtual-bank-of-india.onrender.com/